



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek II.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2025. november 19.

- 1 Legrövidebb utak minden csúcspárra
- 2 Tranzitív lezárt
 - Definíció
 - Példa útvonalakkal
 - Példa mátrix-szal
- 3 Floyd-Warshall
 - Hasonlóság a tranzitív lezárás algoritmussal
 - Floyd–Warshall algoritmus
 - F–W példa

Legrövidebb utak minden csúcspárra

- Feladat: Adott egy *élsúlyozott gráf*, keressük minden csúcspárra a két csúc között lévő legrövidebb utat.
- Ilyen feladat adódik például egy autóstérkép készítésekor is (városok közötti távolság táblázat).
- Irányított gráf esetén megengedünk negatív éleket, negatív összsúlyú köröket azonban nem, irányítatlan esetben negatív éleket sem engedünk meg.
- A feladatot megoldhatjuk a **Dijkstra** vagy a **Bellman-Ford** algoritmussal is, hogy minden csúcsból, elindítjuk az algoritmusokat. De a futási idő költsége n -szeres lesz.

Tanult algoritmussal minden csúcsra

- Dijkstra algoritmus:
 - Ritka gráfon, kupacos minQueue esetén, egy csúcsból elindítva $MT(n, m) = O((n + m) * \log_n)$
 - Minden csúcsból elindítva: $O(n * (n + m) * \log_n)$, feltéve, hogy $m \in \theta(n)$ $MT(n, m) = O(n^2 * \log_n)$
 - Sűrű gráfon, rendezetlen tömböt használva minQueue helyett, egy csúcsból elindítva: $MT(n, m) = \theta(n^2)$ feltéve, hogy $m \in \theta(n^2)$
 - Minden csúcsból elindítva: $MT(n, m) = \theta(n^3)$
- Bellman-Ford algoritmus:
 - Egy csúcsra elindítva: $MT(n, m) = O(n * m)$
 - Ritka gráfon, minden csúcsból elindítva $MT(n, m) = O(n^2 * m) = O(n^3)$, $m \in \theta(n)$ esetén.
 - Sűrű gráfra: $MT(n, m) = O(n^2 * m) = O(n^4)$, mivel $m \in \theta(n^2)$ ilyenkor.

Legrövidebb utak minden csúcspárra

- Egy súlyozott irányított/irányítatlan gráf esetén szeretnénk meghatározni az összes (u, v) csúcspár közötti legrövidebb utat.
- Egy $D/1 : \mathbb{R}_\infty[n, n]$ mátrixot fogunk kiszámítani, ahol
 - $D[i, j]$ az i csúcsból j csúcsba vezető legrövidebb út hossza;
 - vagy $D[i, j] = \infty$, ha nem létezik út i -ből j -be.
- Egy $\pi/1 : \mathbb{N}[n, n]$ mátrixot is kiszámítunk, ahol
 - $\pi[i, j]$ a j csúcs szülőcsúcsa egy i -ből j -be vezető optimális úton, ha $i \neq j$;
 - és $\pi[i, j] = 0$ egyébként.
- A jelölésekben használni fogjuk a $D_{ij} = D[i, j]$ és $\pi_{ij} = \pi[i, j]$ alakokat is.

Tranzitív lezárt

Tranzitív lezárás

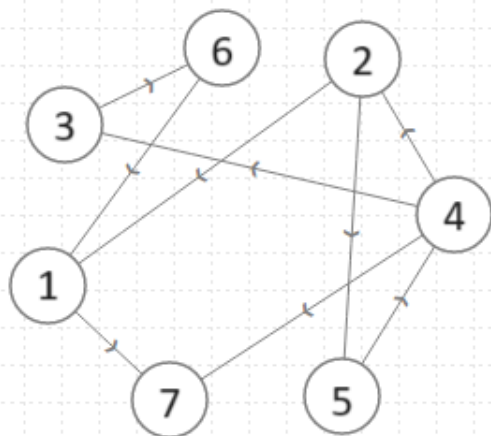
Egy gráf minden (u, v) csúcspárjára meg szeretnénk határozni, hogy létezik-e bármilyen út u -ból v -be. Azt nem vizsgáljuk, hogy mi az út vagy mennyi a hossza. Ehhez a következő fogalmat vezetjük be.

Definíció: Ha $G = (V, E)$ gráf adott, akkor annak **tranzitív lezárása** a $T \subseteq V \times V$ reláció, ahol

$$(u, v) \in T \iff \exists \text{ út } u\text{-től } v\text{-ig a } G \text{ gráfban.}$$

Feltéve, hogy a gráf csúcsait az $1, 2, \dots, n$ indexek azonosítják, a gráfot reprezentálhatjuk a $A/1 : \mathbb{B}[n, n]$ szomszédsági mátrixszal. A tranzitív lezárást pedig a $T/1 : \mathbb{B}[n, n]$ mátrixszal, ahol

$$T[i, j] \iff \exists \text{ út } i\text{-től } j\text{-ig az } A \text{ mátrix által ábrázolt gráfban.}$$



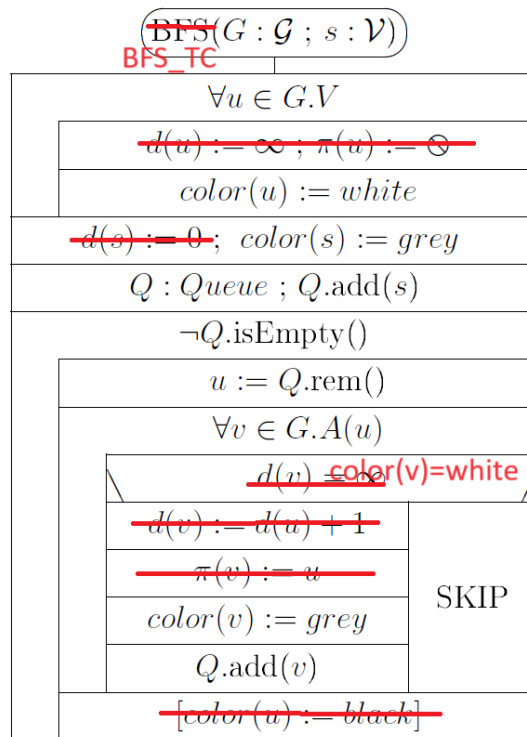
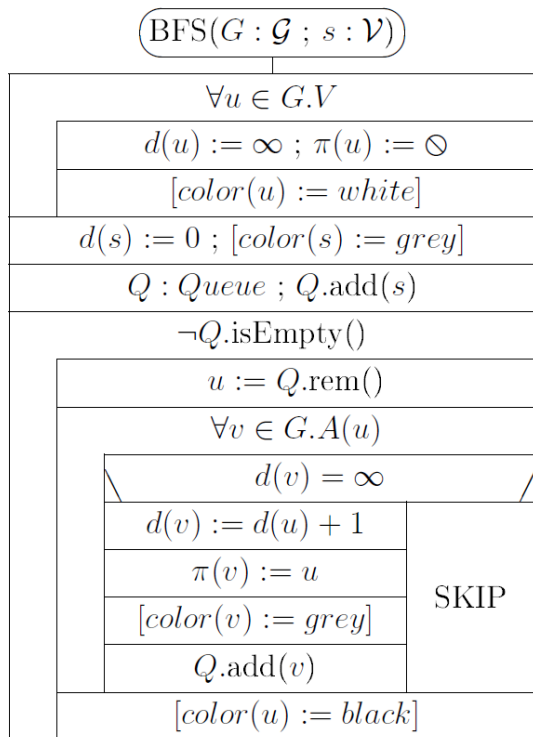
Directed Graph

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure

Hogyan számíthatjuk ki a T mátrixot?

Első megoldás: futtassunk BFS-t minden csúcsból kiindulva. Nem kell sem távolságot, sem szülőt számolnunk, csak az elérhetőséget. BFS-ben egy csúcs elérhető, ha nem fehér.



Ha a BFS_TC algoritmust az i forrásponttal futtatjuk, akkor:

$$T[i, j] \iff \text{color}(j) = \text{grey}$$

így megkapjuk a T mátrix i -edik sorát.

Ha ezt minden i csúcsra végrehajtjuk, akkor megkapjuk a teljes T mátrixot.

Az időbonyolultság ekkor n darab $\Theta(m + n)$ futtatás, vagyis:

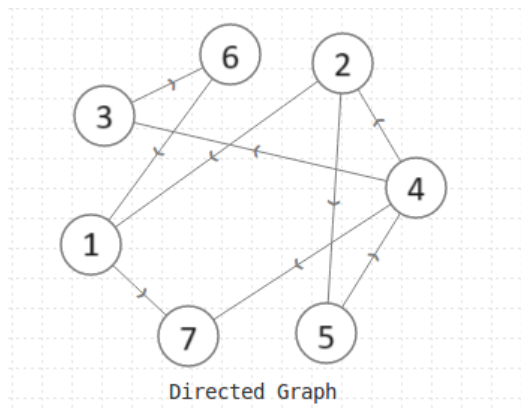
$$\Theta(n(n + m))$$

Ritka gráfoknál ez $\Theta(n^2)$, de sűrű gráfoknál $\Theta(n^3)$.

Másik megoldás a tranzitív lezárás kiszámítására:

Jelölés. $i \overset{k}{\rightsquigarrow} j$ jelentse azt, hogy létezik út i -ből j -be, amely csak $\leq k$ indexű csúcsokat használ köztes csúcsokként.

Tehát az út lehet $\langle i, u_1, u_2, \dots, u_n, j \rangle$ ahol $u_1, u_2, \dots, u_n \leq k$.



Ebben a gráfban nincs $2 \overset{0}{\rightsquigarrow} 3$ út, nincs $2 \overset{1}{\rightsquigarrow} 3$ vagy $2 \overset{4}{\rightsquigarrow} 3$ út sem, de van $2 \overset{5}{\rightsquigarrow} 3$ út. Nincs $3 \overset{5}{\rightsquigarrow} 1$ út, de van $3 \overset{6}{\rightsquigarrow} 1$ út.

Definíció: $T^{(k)} \iff \exists i \overset{k}{\rightsquigarrow} j$, ahol $k \in 0 \dots n$ és $i, j \in 1 \dots n$

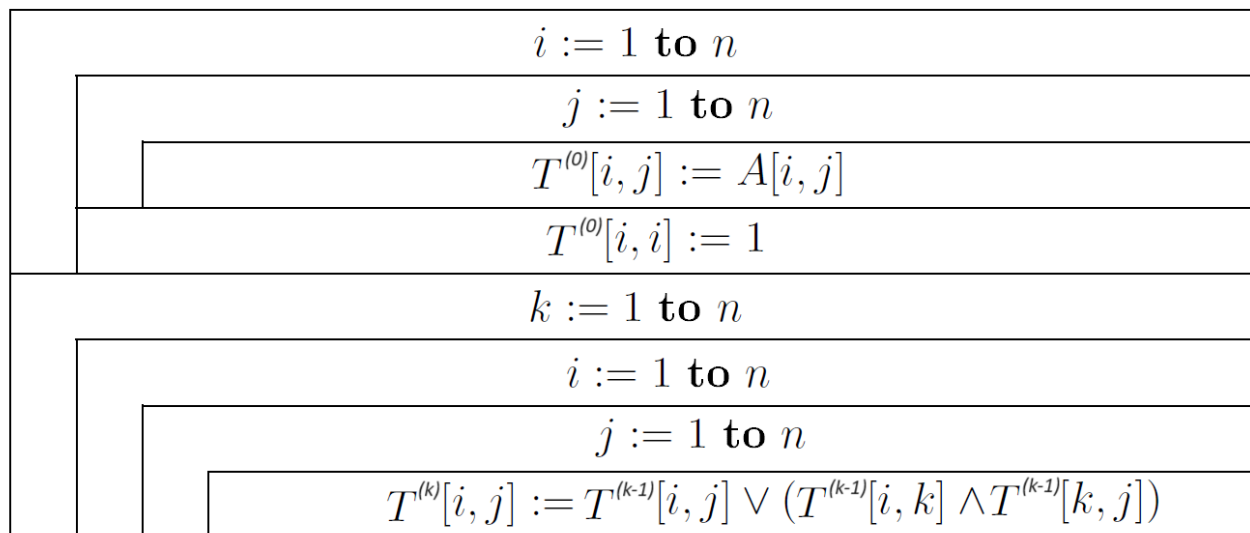
Minden $k = 0, 1, \dots, n$ esetén kiszámítjuk a $T^{(k)}$ mátrixot, és nyilván

$$T^{(n)} = T.$$

Rekurzív összefüggés a T mátrixok között:

- $T_{ij}^{(0)} = A_{ij} \vee (i = j)$ minden $i, j \in 1..n$ esetén
- $T_{ij}^{(k)} = T_{ij}^{(k-1)} \vee (T_{ik}^{(k-1)} \wedge T_{kj}^{(k-1)})$ minden $k, i, j \in 1..n$ esetén

Ez alapján jó algoritmus $T = T^{(n)}$ kiszámítására:

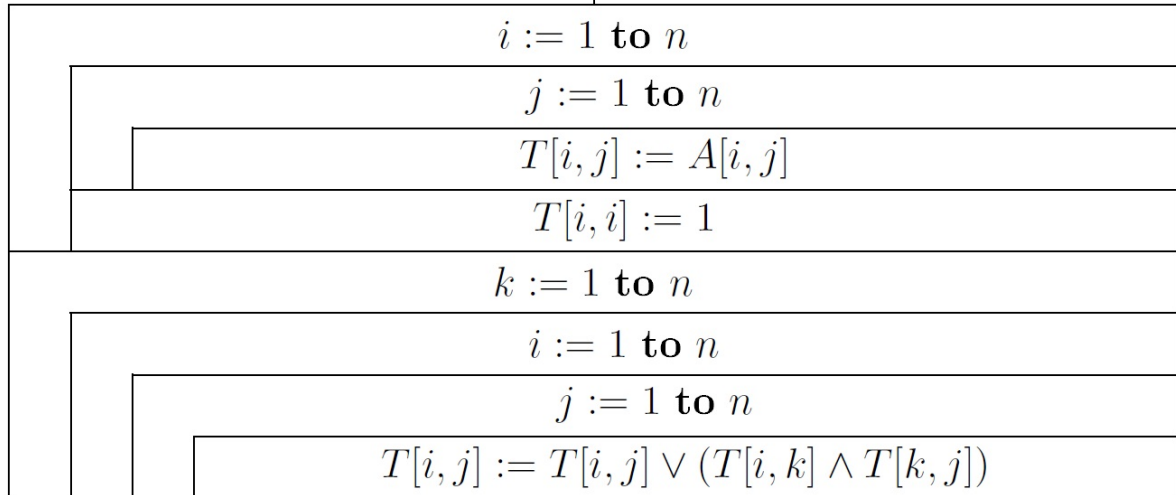


Megfigyelés: A rekurzióból következik:

- $T_{ik}^{(k)} = T_{ik}^{(k-1)}$
- $T_{kj}^{(k)} = T_{kj}^{(k-1)}$

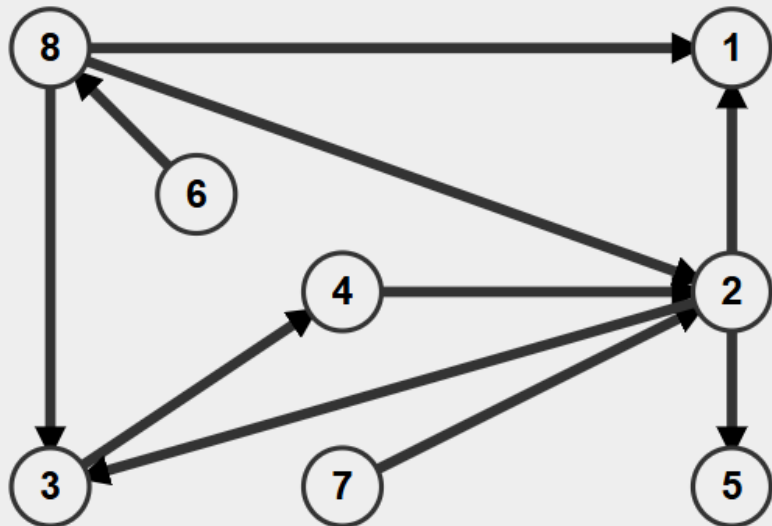
Ez azt jelenti, hogy a k -adik sor és oszlop a $T^{(k)}$ mátrixban megegyezik a megelőző mátrix megfelelő sorával és oszlopával. Ebből következik, hogy az egész számításhoz elegendő egyetlen mátrix T , mert $T_{ij}^{(k)}$ csak $T_{ij}^{(k-1)}$, $T_{ik}^{(k-1)}$ és $T_{kj}^{(k-1)}$ értékétől függ.

TransitiveClosure($A/1, T/1 : \mathbb{B}[n, n]$)



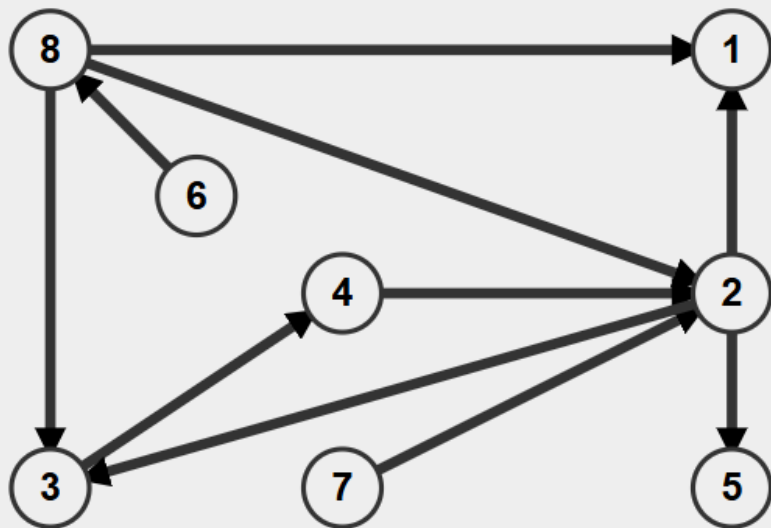
Nyilvánvalóan az algoritmus időbonyolultsága $\Theta(n^3)$.

Példa útvonalakkal (1/3)



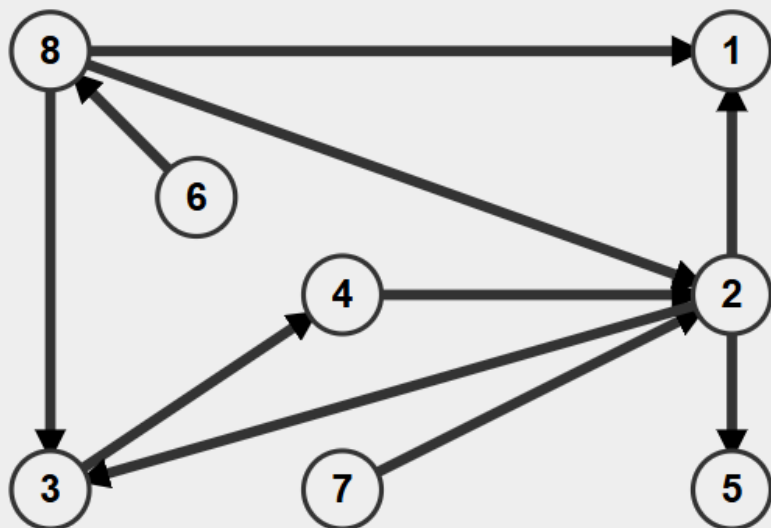
- $K=0$ -> az induló mátrix közvetlen élekkel:
 $P(21), P(25), P(34), P(42), \dots$
- $K=1$ -> nincs változás ((1) nem lehet köztes csúcs)
- $K=2$ -> újonnan felfedezett utak:
 - $P(4..1) = P(42) + P(21)$
 - $P(4..3) = P(42) + P(23)$
 - $P(4..5) = P(42) + P(25)$
 - $P(7..1) = P(72) + P(21)$
 - $P(7..3) = \dots$
 - $P(7..5) = \dots$
 - $P(8..1) = \dots$
 - $P(8..3) = \dots$
 - $P(8..5) = \dots$

Példa útvonalakkal (2/3)

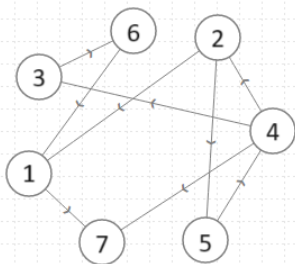


- $K=3 \rightarrow$
 - $P(2..4) = P(23) + P(34)$
 - $P(7..4) = P(7..3) + P(34)$
 - $P(8..4) = P(8..3) + P(34)$
- $K=4 \rightarrow$
 - $P(3..1) = P(34) + P(4..1)$
 $= P(34) + P(42) + P(21)$
 - $P(3..2) = P(34) + P(42)$
 - $P(3..5) = P(34) + P(45)$
 $= P(34) + P(42) + P(25)$
- $K=5 \rightarrow$ nincs változás ((5) nem köztes csúcs)
- $K=6 \rightarrow$ nincs változás ((6) nem köztes csúcs)

Példa útvonalakkal (3/3)



- $K=7 \rightarrow$ nincs változás ((7) nem köztes csúcs)
- $K=8 \rightarrow$
 - $P(6..1) = P(68) + P(81)$
 - $P(6..2) = \dots$
 - $P(6..3) = \dots$
 - $P(6..4) = \dots$
 - $P(6..5) = \dots$



Directed Graph

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

Legrövidebb utak minden csúcspár között

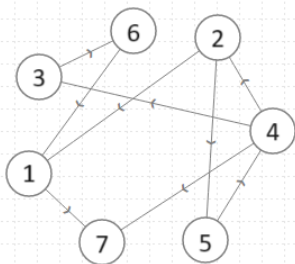
Definíció

Tranzitív lezárt

Példa útvonalakkal

Floyd-Warshall

Példa mátrix-szal



Directed Graph

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure

Floyd-Warshall

Hasonlóság a tranzitív lezárás algoritmussal

TransitiveClosure($A/1, T/1 : \mathbb{B}[n, n]$)

$i := 1$ to n

$j := 1$ to n

$T[i, j] := A[i, j]$

$T[i, i] := 1$

$k := 1$ to n

$i := 1$ to n

$j := 1$ to n

$T[i, j] := T[i, j] \vee (T[i, k] \wedge T[k, j])$

FloydWarshall($A/1, D/1 : \mathbb{R}_\infty[n, n] ; \pi/1 : \mathbb{N}[n, n]$) : $\mathbb{N}$

$i := 1$ to n

$j := 1$ to n

$D[i, j] := A[i, j]$

$i \neq j \wedge A[i, j] < \infty$

$\pi[i, j] := i$

$\pi[i, j] := 0$

$k := 1$ to n

$i := 1$ to n

$j := 1$ to n

$D[i, j] > D[i, k] + D[k, j]$

$D[i, j] := D[i, k] + D[k, j]$

$\pi[i, j] := \pi[k, j]$

$i = j \wedge D[i, i] < 0$

return i // i is in negative cycle

SKIP

return 0 // shortest-path trees computed

Floyd–Warshall algoritmus

A Floyd–Warshall algoritmus mátrixpárok sorozatát állítja elő:

$$(D^{(0)}, \pi^{(0)})$$

$$(D^{(1)}, \pi^{(1)})$$

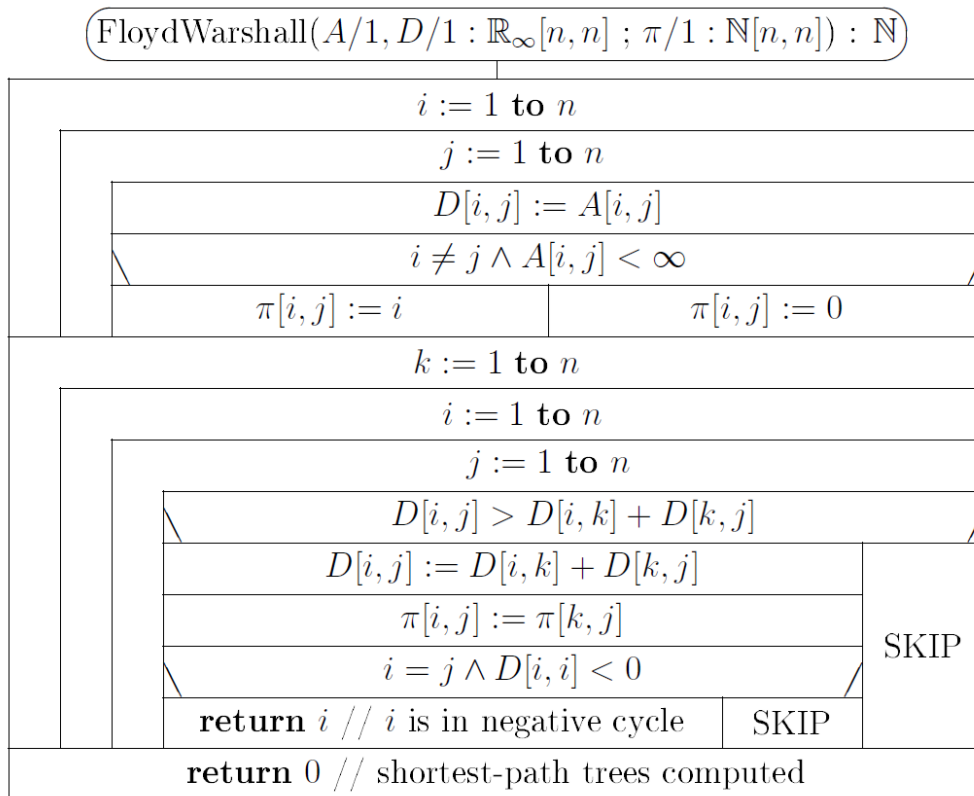
$$(D^{(2)}, \pi^{(2)})$$

...

$$(D^{(n)}, \pi^{(n)})$$

- $D^{(k)}/1 : \mathbb{R}_{\infty}[n, n]$,
- $\pi^{(0)}/1 : \mathbb{N}[n, n]$
- $D^{(0)} = A$, vagyis G szomszédsági mátrixa.
- $D^{(n)} = D$, az összes páros legrövidebb utak mátrixa, és $\pi^{(n)} = \pi$.

A Floyd–Warshall algoritmus



Notation 4.8 $i \overset{k}{\rightsquigarrow}_{\text{opt}} j$ ($k \in 1..n$) is a shortest path from vertex i to vertex j with two constraints:

- On this path, the indices of the vertices between vertex i and vertex j are $\leq k$.
- This path contains no cycle.

Note 4.9

- If $i = j$ then $i \xrightarrow[k]{\text{opt}} j = \langle i \rangle$.
- If $i \neq j$ then $\exists i \xrightarrow[0]{\text{opt}} j = \langle i, j \rangle \iff (i, j)$ is an edge of the graph.
- If $i \neq j \wedge k \in 1..n$, there are two possibilities about path $i \xrightarrow[k]{\text{opt}} j$:

$$i \xrightarrow[k]{\text{opt}} j = i \xrightarrow[k-1]{\text{opt}} j \quad \vee \quad i \xrightarrow[k]{\text{opt}} j = i \xrightarrow[k-1]{\text{opt}} k \xrightarrow[k-1]{\text{opt}} j.$$

A $D^{(k)}$ és $\pi^{(k)}$ mátrixok definíciója

Definition 4.10 $D_{ij}^{(k)} = \begin{cases} w(i \overset{k}{\rightsquigarrow}_{opt} j) & \text{if } i \overset{k}{\rightsquigarrow} j \text{ exists} \\ \infty & \text{if } i \overset{k}{\rightsquigarrow} j \text{ does not exist} \end{cases}$

Definition 4.11

$$\pi_{ij}^{(k)} = \begin{cases} \text{the parent of vertex } j \text{ on a path } i \overset{k}{\rightsquigarrow}_{opt} j, & \text{if } i \neq j \wedge i \overset{k}{\rightsquigarrow} j \text{ exists} \\ 0 & \text{if } i = j \vee i \overset{k}{\rightsquigarrow} j \text{ does not exist} \end{cases}$$

A $D^{(k)}$ és $\pi^{(k)}$ mátrixok tulajdonságai

Property 4.12 *Matrix $D^{(0)}$ is equal to adjacency matrix A of the graph, and matrix $D^{(n)}$ is equal to matrix D to be computed.*

Property 4.13

$$\pi_{ij}^{(0)} = \begin{cases} i & \text{if } i \neq j \wedge (i, j) \text{ is an edge of the graph} \\ 0 & \text{if } i = j \vee (i, j) \text{ is not edge of the graph} \end{cases}$$

And matrix $\pi^{(n)}$ is equal to matrix π to be computed.

Property 4.14 *based on note 4.9, provided that $k \in 1..n$:*

If $D_{ij}^{(k-1)} > D_{ik}^{(k-1)} + D_{kj}^{(k-1)}$

then $D_{ij}^{(k)} = D_{ik}^{(k-1)} + D_{kj}^{(k-1)} \wedge \pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$

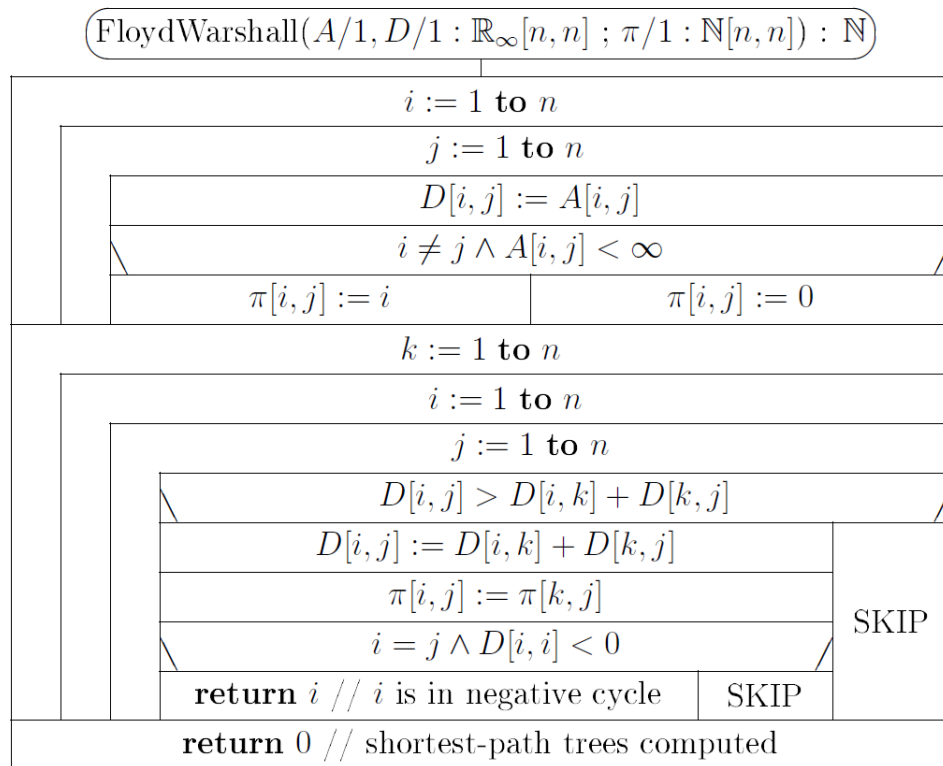
else $D_{ij}^{(k)} = D_{ij}^{(k-1)} \wedge \pi_{ij}^{(k)} = \pi_{ij}^{(k-1)}$

Consequence 4.15 *Provided that $k \in 1..n$:*

$$D_{ik}^{(k)} = D_{ik}^{(k-1)} \wedge \pi_{ik}^{(k)} = \pi_{ik}^{(k-1)} \quad \wedge \quad D_{kj}^{(k)} = D_{kj}^{(k-1)} \wedge \pi_{kj}^{(k)} = \pi_{kj}^{(k-1)}$$

Következmény: az k -adik lépésben az k -adik sor és oszlop nem változik. Ezért nincs szükség külön $D^{(k)}$, $\pi^{(k)}$ mátrixokra minden $k = 1..n$ esetén, a számítás „in place” is elvégezhető.

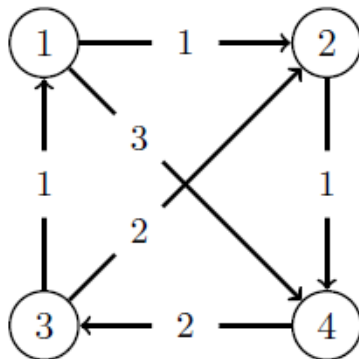
A Floyd–Warshall algoritmus



A Floyd–Warshall algoritmus tulajdonságai

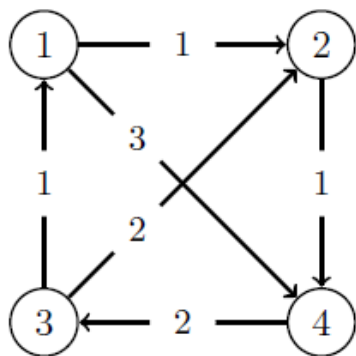
- Az algoritmus helyességének bizonyítása megtalálható a feltöltött előadásjegyzetben.
- Az FW algoritmus időbeli bonyolultsága $O(n^3)$.
- A főátlóban negatív érték jelenik meg akkor és csak akkor, ha a gráf tartalmaz negatív kört. Ha a főátló i -edik eleme negatív, akkor az i csúcs egy negatív körben van.
- A legrövidebb utak fája a $\pi(i, j)$ értékek segítségével visszafejthetők.
- Hasonlítsd össze a Floyd–Warshall algoritmust a korábban látott tranzitív lezárás algoritmussal.
-

F-W példa



$D^{(0)}$	1	2	3	4
1	0	1	∞	3
2	∞	0	∞	1
3	1	2	0	∞
4	∞	∞	2	0

$\pi^{(0)}$	1	2	3	4
1	0	1	0	1
2	0	0	0	2
3	3	3	0	0
4	0	0	4	0

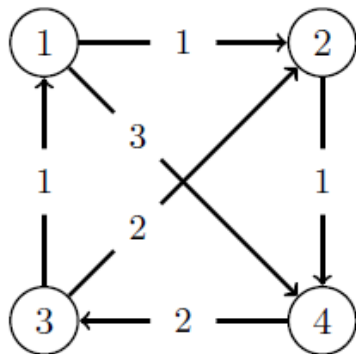


$D^{(0)}$	1	2	3	4
1	0	1	∞	3
2	∞	0	∞	1
3	1	2	0	∞
4	∞	∞	2	0

$\pi^{(0)}$	1	2	3	4
1	0	1	0	1
2	0	0	0	2
3	3	3	0	0
4	0	0	4	0

$D^{(1)}$	1	2	3	4
1	0	1	∞	3
2	∞	0	∞	1
3	1	2	0	4
4	∞	∞	2	0

$\pi^{(1)}$	1	2	3	4
1	0	1	0	1
2	0	0	0	2
3	3	3	0	1
4	0	0	4	0

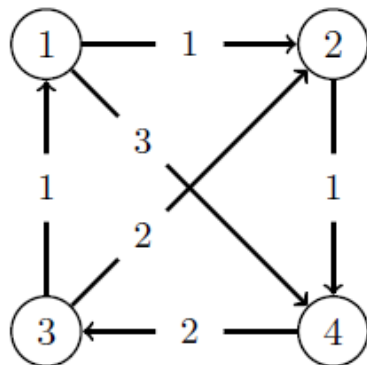


$D^{(1)}$	1	2	3	4
1	0	1	∞	3
2	∞	0	∞	1
3	1	2	0	4
4	∞	∞	2	0

$\pi^{(1)}$	1	2	3	4
1	0	1	0	1
2	0	0	0	2
3	3	3	0	1
4	0	0	4	0

$D^{(2)}$	1	2	3	4
1	0	1	∞	2
2	∞	0	∞	1
3	1	2	0	3
4	∞	∞	2	0

$\pi^{(2)}$	1	2	3	4
1	0	1	0	2
2	0	0	0	2
3	3	3	0	2
4	0	0	4	0

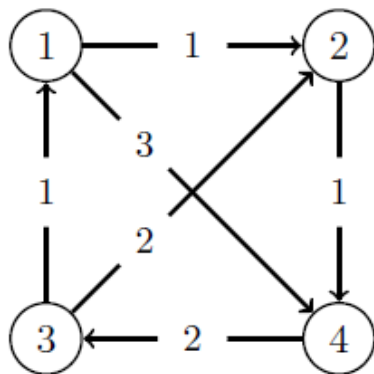


$D^{(2)}$	1	2	3	4
1	0	1	∞	2
2	∞	0	∞	1
3	1	2	0	3
4	∞	∞	2	0

$D^{(3)}$	1	2	3	4
1	0	1	∞	2
2	∞	0	∞	1
3	1	2	0	3
4	3	4	2	0

$\pi^{(2)}$	1	2	3	4
1	0	1	0	2
2	0	0	0	2
3	3	3	0	2
4	0	0	4	0

$\pi^{(3)}$	1	2	3	4
1	0	1	0	2
2	0	0	0	2
3	3	3	0	2
4	3	3	4	0



$D^{(3)}$	1	2	3	4
1	0	1	∞	2
2	∞	0	∞	1
3	1	2	0	3
4	3	4	2	0

$\pi^{(3)}$	1	2	3	4
1				
2				
3	3	3		
4	3	3	4	

$D^{(4)}$	1	2	3	4
1	0	1	4	2
2	4	0	3	1
3	1	2	0	3
4	3	4	2	0

$\pi^{(4)}$	1	2	3	4
1				
2	3			
3	3	3		
4	3	3	4	

Köszönöm a figyelmet!